

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 02 · LECTURE NOTES

Randomness

The foundation everything else stands on.

MODULE OUTCOMES

- Distinguish true, pseudo, and cryptographic randomness.
- Name the correct source on each platform.
- Explain the historical randomness failures and their cause.
- Explain why nonces and keys have different requirements.
- Audit code for randomness misuse.

1 Three kinds of randomness

Kind	The property it has	What it is for
True randomness / entropy source	Physically unpredictable — thermal noise, timing jitter, a hardware instruction.	Seeding the CSPRNG. Rarely used directly, because it is slow and of variable quality.
Ordinary PRNG	Statistically uniform, and completely predictable from its internal state.	Simulation, sampling, games, shuffling for fairness — never anything cryptographic. See the note.
CSPRNG	The next output is unpredictable even to an adversary who has seen all previous output.	Everything cryptographic — keys, nonces, salts, tokens, identifiers that must not be guessable.

The middle row is the trap, and it catches professionals. A Mersenne Twister passes every statistical randomness test that exists — it is uniform, uncorrelated, and has an enormous period — and its entire future and past output is recoverable from 624 consecutive outputs, because that is exactly its state. Statistical uniformity is not unpredictability, and the tests that measure the first say nothing about the second.

THE REQUIREMENT IS UNPREDICTABILITY, NOT UNIFORMITY

An ordinary PRNG produces output that passes every statistical test of randomness — uniform distribution, no autocorrelation, maximal entropy by every measure you can compute — **and is entirely predictable once its internal state is known.**

And the state is recoverable from the output. Observing sufficiently many outputs reconstructs the state exactly, after which every past and future value is known with certainty — not estimated, known.

A CSPRNG adds precisely the property that matters: knowing all previous output gives an adversary no advantage in predicting the next bit, which is exactly the indistinguishability framing of Module 01 §3 applied to a generator rather than to a cipher.

So the test is never 'does this output look random' — it is 'which function produced it', and that is a question about the source code rather than about the bytes, which is why §4's audit is a code review rather than a statistical analysis.

2 The right source

CORRECT

Linux/BSD	getrandom(2), or read /dev/urandom
Windows	BCryptGenRandom
macOS/iOS	SecRandomCopyBytes / arc4random_buf
Python	secrets.*, or os.urandom
Java	SecureRandom
Go	crypto/rand
Node	crypto.randomBytes
libsodium	randombytes_buf

NEVER FOR ANYTHING CRYPTOGRAPHIC

rand(), random(), Math.random()
 java.util.Random
 Python's random module
 any PRNG you seeded yourself
 the current time, a PID, a counter, a UUIDv1

AND /dev/random vs /dev/urandom: on any modern Linux kernel, use urandom or getrandom. The folklore about urandom being cryptographically weaker is obsolete.

The urandom folklore persists and is wrong on every current kernel. Blocking on /dev/random buys no additional security and causes real availability failures — services that hang at boot waiting for entropy that is already sufficient. **The one genuine edge case is a freshly provisioned virtual machine or embedded device generating long-term keys before any entropy has accumulated**, which is §3's third failure and is addressed by getrandom(2) blocking only until the pool is initialised once.

3 How it fails

- **Seeding from a predictable value.** The current time, a process identifier, or a counter — which reduces an apparently 128-bit key space to something exhaustively searchable in seconds.
- **A patch that destroys the entropy pool.** One distribution's well-intentioned removal of a line that used uninitialised memory reduced the effective key space to a few tens of thousands of possibilities — and the code continued to work perfectly for two years.
- **Generating long-term keys at first boot,** before any entropy has accumulated — **which produced large populations of embedded devices and virtual machines sharing identical host keys**, discoverable by anyone who scanned for them.
- **A hardware source trusted without mixing.** Which is exactly why every serious platform mixes hardware instruction output into a software pool rather than returning it directly — a defective or backdoored hardware source then degrades rather than destroys the output.
- **And forking.** A process that forks after seeding gives both children an identical stream unless the library detects it — which produced duplicate session keys in more than one widely deployed server. **Every one of these produced working software**, which is the lesson: **randomness failures are silent, the keys generate, the encryption works, and the key space is tiny.**

RANDOMNESS FAILURES ARE CATASTROPHIC, SILENT, AND RETROACTIVE

Catastrophic, because every key, nonce, salt, and token derived from a weak source is weak — and the strength of the cipher using it is entirely irrelevant. AES-256 with a key drawn from 2^{16} possibilities offers 16 bits of security.

Silent, because nothing fails. There is no error, no exception, no test failure, and no observable difference in behaviour until somebody exploits it — which may be years after deployment, and which is how the historical examples in §3 all went undetected for extended periods.

And retroactive: keys generated during the weak period remain weak after the bug is fixed, so remediation means identifying, regenerating, and rotating every affected key and every artefact derived from one — **which is why Module 10's rotation procedure needs to exist before you need it** (and CSCE 701 Module 07 §3).

So the correct posture is to use the platform source and never touch it again. This is the one place in all of cryptography where there is genuinely nothing to tune, nothing to optimise, and no legitimate reason to be clever — which makes it the easiest rule in the course to follow and one of the most frequently broken.

4 Nonces, keys, and what each needs

Value	What it actually requires	Consequence of getting it wrong
Key	Unpredictable and secret.	Total loss of confidentiality and authenticity.

Value	What it actually requires	Consequence of getting it wrong
Nonce / IV	Unique per key. It need not be secret and, for most modes, need not be unpredictable.	Severe and mode-dependent — catastrophic for counter-mode constructions (Module 11 §2).
Salt	Unique. It need not be secret or unpredictable.	Precomputation across users becomes possible (Module 09).
Session token	Unpredictable.	Session hijacking (CSCE 701 Module 02 §3).
Password reset token	Unpredictable, single-use, and expiring.	Account takeover — and the single-use requirement is the one usually omitted.

A nonce requires uniqueness and not secrecy, which is why it ships in the clear alongside the ciphertext and why a counter is a perfectly valid nonce wherever you can genuinely guarantee the counter never repeats for a given key — the difficulty being that distributed systems and restarts make that guarantee much harder than it sounds (CSCE 678 Module 05).

- Any import of a non-cryptographic random module in a file that handles keys, tokens, or identifiers that must not be guessable. This single grep finds real defects.
- Any explicit seeding call. A correct CSPRNG is never seeded by application code, so the presence of a seed call is itself a defect signal regardless of what it seeds from.
- Tokens constructed from a timestamp, a counter, a hash of predictable inputs, or a sequential database identifier — all of which are unique and none of which are unguessable.
- UUID version 1 used as a secret. It encodes a timestamp and a network address and is designed for uniqueness, not unguessability — use version 4 from a CSPRNG, or better, secrets.token_urlsafe.
- And key generation at install time or first boot, which is §3's embedded-device failure arriving in your own provisioning script. 'Unique' and 'unguessable' are different requirements, and UUID conflates them in the mind of nearly every developer who reaches for one.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Aumasson — Serious Cryptography, chapter 2	The whole module, with more detail on the entropy pool design and the hardware source question.
Heninger et al. — Mining Your Ps and Qs (free)	§3's third failure, measured at internet scale — the study that found hundreds of thousands of devices with shared or factorable keys.
Myths about /dev/urandom (free)	§2's folklore, dismantled carefully — read it once and stop worrying about this.

Resource	What it covers for this module
NIST SP 800-90A/B/C (free)	The standards for CSPRNG construction and entropy assessment — reference rather than reading, but worth knowing exists.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Generate 1000 values from your language's default random module** and from its CSPRNG, and confirm both pass a basic statistical test.
- 2 **Recover a Mersenne Twister's state** from its output, and predict the next value.
- 3 **Explain why that is possible** for one and not the other.
- 4 **Grep a codebase for non-cryptographic random imports** and classify each use as safe or not.
- 5 **Find any explicit seeding** and determine what it seeds from.
- 6 **Find how your platform's CSPRNG behaves at early boot**, and whether it blocks.
- 7 **Write the five-row requirements table from memory** and check it.
- 8 **Audit your own token generation** for every token type your system issues.
- 9 **Check whether any of your tokens are single-use** and expiring where they should be.
- 10 **Determine what your application would have to do** to rotate every key after a randomness failure.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the three kinds and what each is for.
- 2 Why is a Mersenne Twister dangerous despite passing every statistical test?
- 3 What property does a CSPRNG add, and how does it relate to Module 01 §3?
- 4 Name the correct source on three platforms, and four things never to use.
- 5 What is the truth about `/dev/random` versus `/dev/urandom`?
- 6 Give five historical randomness failures.
- 7 Why are randomness failures silent, and why retroactive?
- 8 What does a nonce require that a key does not, and vice versa?
- 9 Why can a counter be a valid nonce, and what makes that hard?
- 10 Why is UUIDv1 unsuitable as a secret?